

- Caylent — Principal Cloud Architect | Whiteboarding Round Preparation
 - Table of Contents
 - 1. What Caylent Is Really Evaluating {#1-what-caylent-is-really-evaluating}
 - Lens 1: Enterprise Migrations & Modernization
 - Lens 2: Cloud-Native Scale & AI-Driven Operations
 - The 5 Qualities They Are Scoring
 - 2. The 4 Cornerstones to Ace the Session {#2-the-4-cornerstones}
 - Cornerstone 1: Own the Ambiguity (Don't Draw Too Quickly)
 - Cornerstone 2: Think in Multi-Account Landing Zones
 - Cornerstone 3: Emphasize Modernization Over "Lift and Shift"
 - Cornerstone 4: Talk "Day 2 Operations" & Infrastructure as Code
 - 3. Your Whiteboard Drawing Framework {#3-whiteboard-drawing-framework}
 - Board Layout (practice this physical layout)
 - Drawing Sequence (practice this order)
 - Labeling Convention
 - 4. The STARS Execution Method {#4-stars-execution-method}
 - 5. Scenario 1: Enterprise Migration to AWS {#5-scenario-1-enterprise-migration}
 - Prompt: "A client has a legacy on-premises application stack — 50 VMs, Oracle database, Windows/.NET services. They want to move to AWS within 6 months. Design the migration strategy."
 - Step 1: Own the Ambiguity (5-10 minutes of questions)
 - Step 2: Draw the Account Foundation (Cornerstone 2)
 - Step 3: Migration Strategy — The 7 R's (draw this)
 - Step 4: Phased Migration Architecture (main board area)
 - Step 5: Data Migration Detail (draw this clearly)
 - Key Talking Points (say these out loud)
 - 6. Scenario 2: Serverless-First Architecture on AWS {#6-scenario-2-serverless-first-architecture}
 - Prompt: "Design a serverless API backend for a fintech client that processes 10,000 transactions per minute with sub-100ms latency."
 - Discovery Questions (Cornerstone 1)
 - Architecture (draw left-to-right)
 - Key Design Decisions (Trade-Off Table)
 - Performance Budget (draw this)
 - Day 2 Operations (Cornerstone 4)
 - Cost Estimate

- 7. Scenario 3: Multi-Account Landing Zone Design {#7-scenario-3-multi-account-landing-zone}
 - Prompt: "Design an AWS multi-account landing zone for an enterprise with 200 developers, 50 applications, and strict compliance requirements."
 - Discovery Questions
 - Architecture (this is Cornerstone 2 in full)
 - Service Control Policies (SCPs) — Draw Key Ones
 - Network Architecture (Hub-and-Spoke via Transit Gateway)
 - Account Vending Machine (Day 2 Operations)
- 8. Scenario 4: Real-Time Data Platform Architecture {#8-scenario-4-real-time-data-platform}
 - Prompt: "A client needs a real-time data analytics platform. They have 500GB of streaming data per day from IoT devices and need dashboards with <5 second refresh."
 - Discovery Questions
 - Architecture (Lambda Architecture — Hot + Cold Paths)
 - Key Design Decisions
 - Day 2 Operations
- 9. Scenario 5: Modernization — Monolith to Microservices on EKS {#9-scenario-5-monolith-to-microservices}
 - Prompt: "A client has a Java monolith serving 5M users. They want to modernize to microservices on EKS. Design the modernization strategy and target architecture."
 - Discovery Questions
 - Strangler Fig Strategy (Cornerstone 3 — Modernization)
 - Target EKS Architecture
 - Database Per Service
- 10. Scenario 6: DR & High Availability Architecture {#10-scenario-6-dr-and-high-availability}
 - Prompt: "Design a DR strategy for a banking client with RPO < 1 minute and RTO < 15 minutes."
 - Discovery Questions
 - Multi-Region Architecture
 - RTO/RPO Analysis
 - Day 2: DR Drill Automation
- 11. Scenario 7: CI/CD & Platform Engineering {#11-scenario-7-cicd-and-platform-engineering}

- Prompt: "Design a CI/CD platform for 50 development teams deploying to EKS, with security gates and compliance controls."
- Discovery Questions
- Platform Architecture (Cornerstone 4 — this IS Day 2 Operations)
- Security Gate Model
- 12. Scenario 8: Security Architecture & Compliance {#12-scenario-8-security-architecture}
 - Prompt: "Design a security architecture for a healthcare client on AWS that must comply with HIPAA."
 - Discovery Questions
 - Defense-in-Depth Architecture (6 Layers)
 - HIPAA-Specific Controls
- 13. Scenario 9: Cost Optimization & FinOps {#13-scenario-9-cost-optimization}
 - Prompt: "A client's AWS bill jumped from 500K/month. How would you investigate and optimize?"
 - Investigation Framework
 - FinOps Operating Model (draw this)
- 14. Scenario 10: Observability at Enterprise Scale {#14-scenario-10-observability}
 - Prompt: "Design an observability strategy for 50 microservices on EKS."
 - Three Pillars Architecture
 - Grafana Dashboard Hierarchy
- 15. Scenario 11: GenAI / Modern Data Infrastructure on AWS {#15-scenario-11-genai-data-infrastructure}
 - Prompt: "Design a GenAI-powered platform on AWS — the client wants to build internal AI applications using foundation models, with a focus on RAG (Retrieval-Augmented Generation) and custom model fine-tuning."
 - Discovery Questions
 - Architecture: GenAI Platform on AWS
 - Key Design Decisions
 - Account Structure for GenAI
 - Day 2: AI Ops
- 16. Caylent-Specific & Consulting Questions {#16-caylent-specific-questions}
 - Q1: Why Caylent? What attracts you to a consulting role at an AWS Premier Partner?
 - Q2: Describe a time you led a discovery session with a client who had conflicting requirements
 - Q3: How do you handle scope creep in a consulting engagement?

- Q4: How do you estimate effort for an architecture engagement?
- Q5: Describe your experience with pre-sales support
- 17. Stakeholder & Leadership Scenarios {#17-stakeholder-leadership-scenarios}
 - Q6: A client's team is resistant to adopting IaC (Terraform). How do you drive adoption?
 - Q7: How do you mentor architects and engineers?
 - Q8: Tell me about a time you made a wrong architectural decision
- 18. Technical Deep-Dive Rapid Fire {#18-technical-deep-dive-rapid-fire}
 - AWS Services
 - IaC & DevOps
 - Observability
- 19. Questions to Ask the Interviewer {#19-questions-to-ask}
 - About the Role
 - About Caylent's AI/Ops Focus
 - About Growth & Culture
 - About Delivery
- Quick Reference: What to Say at Key Moments
 - Opening (when they give you the prompt)
 - Before Drawing
 - Making a Technology Choice
 - Addressing Day 2
 - Closing

Caylent — Principal Cloud Architect | Whiteboarding Round Preparation

Company: Bot Consulting – Caylent (Premier AWS Partner)

Role: Principal Cloud Architect

Round: 2nd Round — 1.5 Hour Whiteboarding Session

Shift: 2-11 PM IST

Prepared for: Pushparaj Naik | 22+ Years Experience

Table of Contents

1. What Caylent Is Really Evaluating
 2. The 4 Cornerstones to Ace the Session
 3. Your Whiteboard Drawing Framework
 4. The STARS Execution Method
 5. Scenario 1: Enterprise Migration to AWS
 6. Scenario 2: Serverless-First Architecture on AWS
 7. Scenario 3: Multi-Account Landing Zone Design
 8. Scenario 4: Real-Time Data Platform Architecture
 9. Scenario 5: Modernization — Monolith to Microservices on EKS
 10. Scenario 6: DR & High Availability Architecture
 11. Scenario 7: CI/CD & Platform Engineering
 12. Scenario 8: Security Architecture & Compliance
 13. Scenario 9: Cost Optimization & FinOps
 14. Scenario 10: Observability at Enterprise Scale
 15. Scenario 11: GenAI / Modern Data Infrastructure on AWS
 16. Caylent-Specific & Consulting Questions
 17. Stakeholder & Leadership Scenarios
 18. Technical Deep-Dive Rapid Fire
 19. Questions to Ask the Interviewer
-

1. What Caylent Is Really Evaluating {#1-what-caylent-is-really-evaluating}

Critical Insight: At the Principal level, a whiteboarding round isn't just about drawing boxes and arrows. They aren't checking if you know AWS services — they are evaluating **how you handle ambiguity, guide a customer through complex trade-offs, and design for real-world business constraints.**

Because Caylent is a **premier AWS consulting partner**, they evaluate your architectural skills through **two main lenses**:

Lens 1: Enterprise Migrations & Modernization

This is Caylent's bread and butter. They want to see:

- Can you take a vague "move us to the cloud" prompt and decompose it into a structured, phased approach?
- Do you instinctively think in **multi-account landing zones**, not single-account toy architectures?
- Can you articulate why **modernization** (EKS, serverless) is more valuable than simple lift-and-shift — while acknowledging when lift-and-shift is the right starting point?
- Do you address **Day 2 reality** — who operates this at 2 AM, how does IaC keep it from drifting, how does the team troubleshoot?

Lens 2: Cloud-Native Scale & AI-Driven Operations

Caylent is increasingly focused on AI-driven cloud operations. They want to see:

- Event-driven architectures that **self-heal** (EventBridge → Lambda auto-remediation)
- Observability that goes beyond dashboards to **automated response**
- GenAI/ML workloads on AWS (SageMaker, Bedrock, EKS with GPU nodes)
- Modern data platforms that combine streaming, lake, and AI inference

The 5 Qualities They Are Scoring

Quality	What They See	What Separates Good from Great
Structured Thinking	Do you follow a framework?	Great: You slow down, ask questions, then draw. Good: You jump to drawing.
Client Empathy	Do you speak as a consultant?	Great: "The client's RTO requirement tells us..." Good: "I would use Aurora..."
Trade-Off Articulation	Can you weigh options?	Great: "I'm choosing X over Y because Z constraint." Good: "X is better."
AWS Depth	Do you know services deeply?	Great: You mention Karpenter, DAX, Control Tower specifics. Good: You say "EKS, DynamoDB."
Delivery Leadership	Can you lead teams?	Great: You discuss phased rollout, team skills, change management. Good: You only

2. The 4 Cornerstones to Ace the Session {#2-the-4-cornerstones}

These are your strategic pillars. Every answer you give should touch at least 2-3 of these.

Cornerstone 1: Own the Ambiguity (Don't Draw Too Quickly)

Rule: The interviewer will purposefully give you a vague prompt. This is a test. Do NOT start drawing immediately.

Spend the first 5-10 minutes asking discovery questions. Treat the interviewer like a client.

DISCOVERY QUESTION CATEGORIES:

Business Context:

"What's driving this initiative – cost, scale, compliance, datacenter exit?"

"What's the timeline expectation? Are we talking 3 months or 12 months?"

"Who are the stakeholders? CTO? VP Eng? Compliance team?"

Non-Functional Requirements:

"What's your RTO and RPO? This fundamentally changes the architecture."

"What are the peak traffic patterns? Steady state vs bursty?"

"Any compliance frameworks – PCI-DSS, HIPAA, SOC2, FedRAMP?"

Operational Reality:

"What's the team's current skill set? Kubernetes experience? IaC adoption?"

"How frequently do you deploy today? Weekly? Daily? Multiple times/day?"

"What does your on-call rotation look like?"

Integration & Constraints:

"Are there on-prem systems that must remain and integrate?"

"Any vendor contracts or licensing that constrain decisions (e.g., Oracle, VMware)?"

"What's the budget range – are we optimizing for speed, cost, or capability?"

Why this works at Caylent: Premier consulting partners bill for discovery. If you jump straight to drawing, you're showing you'd skip the most valuable phase of an engagement. A Principal Architect who asks great questions wins client trust before writing a single line of IaC.

Transition phrase after discovery:

"Okay, based on what you've told me — [summarize 3 key constraints] — let me sketch out an approach. I'll start with the foundation and work through to the application layer."

Cornerstone 2: Think in Multi-Account Landing Zones

Rule: NEVER design a system that sits in a single AWS account. This is consulting 101.

Every whiteboard design should start with the account structure:

```
AWS Organizations
├── Management Account (billing, org policies, SS0)
├── Security OU
│   ├── Log Archive Account (CloudTrail, Config, VPC Flow Logs)
│   └── Security Audit Account (GuardDuty delegated admin, SecurityHub)
├── Infrastructure OU
│   ├── Network Hub Account (Transit Gateway, Direct Connect, DNS)
│   └── Shared Services Account (CI/CD runners, ECR, artifact repos)
├── Workload OU
│   ├── Dev Accounts (per-team or per-application)
│   ├── Staging Accounts
│   └── Production Accounts
└── Sandbox OU (developer experimentation, auto-nuke after 7 days)

Governance:
- AWS Control Tower for guardrails
- IAM Identity Center (formerly AWS SS0) for centralized access
- SCPs enforcing region restrictions, security baseline
- Service Catalog / Account Factory for Terraform (AFT) for vending
```

How to mention this naturally: When you start any whiteboard, say:

"Before I draw the application architecture, let me quickly establish the account structure. At a consulting partner like Caylent, this is the foundation we'd build first — because everything else inherits from it."

Then draw the account tree in a corner of the board, and reference it as you place services:

"This EKS cluster lives in the Production workload account, pulling images from ECR in the Shared Services account. All logs flow to the Log Archive account."

Cornerstone 3: Emphasize Modernization Over "Lift and Shift"

Rule: While simple migrations happen, consulting partners win by modernizing workloads. Show you can design both paths and articulate the trade-off.

Always present a spectrum, not a binary choice:

MODERNIZATION SPECTRUM (present this on the board):

Effort →

Rehost (Lift-Shift)	Replatform (Lift-Optimize)	Refactor (Re-architect)
EC2 + AMI MGN / VM Import	EC2 → RDS Managed services	EKS / Fargate Serverless (Lambda)
Risk: Low Value: Low Speed: Fast OpEx: High	Risk: Medium Value: Medium Speed: Medium OpEx: Medium	Risk: Higher Value: Highest Speed: Slower OpEx: Lowest
"Good for tight deadlines and low-skill teams"	"Sweet spot for most enterprise migrations"	"Right choice when the team has skills and timeline allows"

Containerization path (be ready to go deep):

- Amazon EKS with Fargate (serverless pods) or managed node groups
- **Karpenter** for cluster autoscaling (mention this by name — it's cutting edge and Caylent will notice)

- Service mesh: Istio or AWS App Mesh for mTLS, traffic management
- GitOps: ArgoCD for declarative deployment
- Progressive delivery: Argo Rollouts for canary/blue-green

Serverless path (contrast it explicitly):

- "If the workload is event-driven with variable traffic, I'd contrast the container approach with a serverless architecture using Lambda, EventBridge, and DynamoDB. The trade-off is: less operational overhead but more coupling to AWS services and potential cold-start concerns."

Trade-off speaking pattern (memorize this):

"I'm choosing RDS PostgreSQL here because the customer's schema requires complex relational joins, and their team lacks NoSQL modeling expertise — so this reduces migration risk. If they had simple key-value access patterns, I'd push for DynamoDB for the latency and scaling characteristics."

Cornerstone 4: Talk "Day 2 Operations" & Infrastructure as Code

Rule: A great architecture is useless if it's a nightmare to manage. Principals win by showing how the infrastructure will **live and breathe** after the consulting engagement ends.

IaC Strategy (mention in every scenario):

Infrastructure as Code Approach:

Tool Selection (present trade-off):

Terraform (HCL)

vs

AWS CDK (TypeScript/Python)

Multi-cloud portable

Huge module ecosystem

State management required

Most enterprise teams know it

AWS-only, tighter integration

Constructs = higher abstraction

CloudFormation under the hood

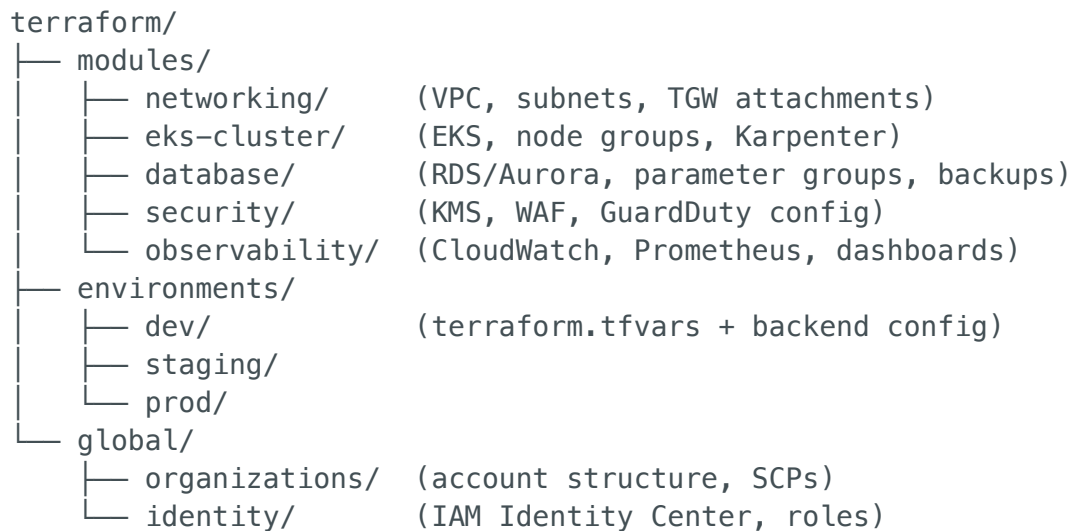
Better for complex logic/loops

"I'd recommend Terraform for this client because [their team already uses it /

they have multi-cloud aspirations / it's the industry standard for consulting

handoffs]."

Modularization Strategy:



Observability (the "how do we know it's healthy" layer):

Observability Stack:

Logs: Fluent Bit (on EKS) → CloudWatch Logs → CloudWatch Insights
 OR → OpenSearch for full-text search

Metrics: ADOT Collector → Amazon Managed Prometheus (AMP)
 → Amazon Managed Grafana dashboards

Traces: AWS X-Ray / ADOT → distributed trace visualization

Alerting: CloudWatch Alarms → SNS → PagerDuty / Slack
 SLO-based: burn rate alerting (not raw thresholds)

AI-Driven Cloud Operations (Caylent's current engineering focus — mentioning this will resonate):

AUTOMATED REMEDIATION PATTERNS:

Pattern 1: Self-Healing Infrastructure

CloudWatch Alarm (CPU > 90% for 5 min)

→ EventBridge Rule

→ Lambda function

→ Scale EKS node group / add Karpenter provisioner capacity

→ Log remediation action to CloudTrail

→ Notify team via Slack

Pattern 2: Security Auto-Response

GuardDuty Finding (High Severity)

→ EventBridge Rule

→ Step Functions workflow:

1. Isolate compromised instance (remove all SGs, add quarantine SG)

2. Snapshot EBS volumes (forensics)

3. Create JIRA ticket
4. Page on-call security engineer
5. Disable compromised IAM credentials

Pattern 3: Compliance Drift Auto-Remediation

AWS Config Rule (S3 bucket not encrypted)

- EventBridge
- Lambda → Enable SSE-KMS on the bucket
- SNS → Notify compliance team
- Config records remediation in compliance timeline

"At a partner like Caylent, I'd build these remediation patterns as reusable modules – so every client engagement gets automated Day 2 operations out of the box, not just a pretty architecture diagram."

3. Your Whiteboard Drawing Framework {#3-whiteboard-drawing-framework}

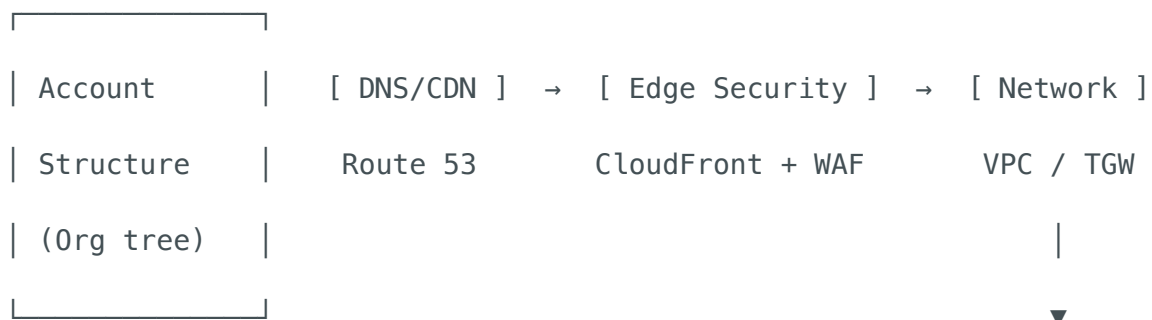
Board Layout (practice this physical layout)

When you start drawing, keep a **clean, left-to-right logical flow** so the interviewer can easily follow your story:

BOARD LAYOUT:

TOP-LEFT CORNER:

MAIN AREA (left-to-right flow):



BOTTOM-LEFT:

EKS / Fargate

RDS / DynamoDB

Fargate for system pods | ← Cost optimization detail

BAD (too vague):

Container
Service

4. The STARS Execution Method {#4-stars-execution-method}

Use this for every scenario as your mental checklist:

S – Scope & Requirements (5–10 minutes)

"Let me clarify the business context and non-functional requirements first..."

Ask: Users? Geography? SLAs? Compliance? Budget? Timeline? Team skills?

★ This is Cornerstone 1 – Own the Ambiguity

★ WRITE DOWN the key NFRs on the board corner

T – Tradeoff Analysis (3–5 minutes)

"There are 2–3 approaches here. Let me walk through the tradeoffs..."

Present options, compare on: risk, cost, timeline, team capability

Recommend ONE with clear reasoning

★ This is Cornerstone 3 – Always speak in trade-offs

A – Architecture Design (15–20 minutes)

Draw the architecture on the whiteboard

Start with account structure → user/client → work inward through layers

Label every component with the AWS service name

★ This is Cornerstone 2 – Always show multi-account structure

R – Resilience & Security (5–10 minutes)

"Let me address failure modes and security..."

Multi-AZ, DR strategy, encryption, IAM, network segmentation

Reference the Security account from your org structure

S – Scalability & Operations (5–10 minutes)

"Here's how this scales and how we operate it Day 2..."

Auto-scaling, observability, CI/CD, IaC strategy, cost implications

★ This is Cornerstone 4 – Day 2 Operations

★ Mention AI-driven auto-remediation patterns

5. Scenario 1: Enterprise Migration to AWS {#5-scenario-1-enterprise-migration}

Prompt: "A client has a legacy on-premises application stack — 50 VMs, Oracle database, Windows/.NET services. They want to move to AWS within 6 months. Design the migration strategy."

 **This is the most likely prompt you'll receive.** Caylent does this daily. Nail this one.

Step 1: Own the Ambiguity (5-10 minutes of questions)

Say out loud:

"Before I start designing, I need to understand the business context. Let me ask a few discovery questions — I'd do the same in a real client engagement."

Discovery Questions to Ask:

Business Driver:

"What's driving the migration? Datacenter lease expiring? Cost reduction? Scalability needs? Or a strategic decision to go cloud-first?"

SLAs & Availability:

"What's the current SLA? What RTO and RPO does the business require?"

"Is there a maintenance window we can use for cutover, or is this 24/7?"

Compliance:

"Any compliance requirements — HIPAA, PCI-DSS, SOC2? This changes everything from encryption to network architecture."

Team & Skills:

"What's the team's AWS skill level? Do they have IaC experience? This affects whether we rehost first or go straight to modernization."

Oracle Licensing:

"What's the Oracle licensing situation? BYOL? Is there appetite to migrate off Oracle to save licensing costs?"

Integration:

"Are there on-prem systems that must remain? We'll need hybrid connectivity."

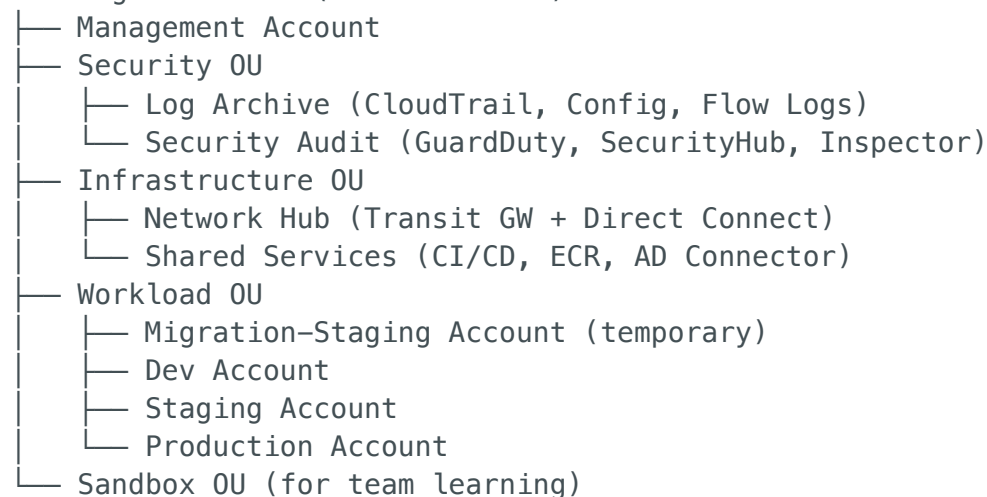
Transition phrase:

"Based on what you've told me — tight 6-month timeline, Oracle dependencies, and a team that's new to AWS — I'd recommend a phased approach: establish the landing zone first, then rehost for speed, with a modernization roadmap for post-migration. Let me draw this out."

Step 2: Draw the Account Foundation (Cornerstone 2)

Draw in top-left corner:

AWS Organizations (Control Tower)



Identity: IAM Identity Center → AD Connector → On-prem Active Directory

Governance: SCPs (region lock, prevent security disabling)

Account Vending: Control Tower Account Factory for Terraform (AFT)

Step 3: Migration Strategy — The 7 R's (draw this)

AWS 7 R's Migration Strategy	
Rehost	Lift-and-shift VMs to EC2 (AWS MGN)
Replatform	Move to managed services (RDS for Oracle)
Repurchase	Replace with SaaS (e.g., on-prem AD → SaaS)
Refactor	Re-architect to cloud-native (ECS/EKS)
Retire	Decommission unused applications
Retain	Keep on-prem (mainframe, regulatory)
Relocate	VMware Cloud on AWS (fastest)

TRADE-OFF ARTICULATION:

"For this 6-month timeline with a team new to AWS, I'd recommend Rehost-first for 80% of workloads, Replatform for the database, and flag 2-3 candidates for Refactoring in Phase 2. The reason is: rehosting de-risks the migration – we get out of the datacenter on time, then modernize iteratively once the team builds AWS confidence."

Step 4: Phased Migration Architecture (main board area)

Phase 1 (Month 1-2): FOUNDATION – "This is where consulting partners add the most value"

- └─ Landing Zone (Control Tower + AFT)
 - └─ Account structure (as drawn above)
 - └─ SCPs: region lock, prevent security disabling, tagging enforcement
 - └─ IAM Identity Center: SSO + MFA → on-prem AD via AD Connector
- └─ Networking
 - └─ Direct Connect (primary) + Site-to-Site VPN (failover)
 - └─ Transit Gateway (hub-and-spoke topology)
 - └─ Network Firewall in Inspection VPC (stateful IDS/IPS)
 - └─ Route 53 Private Hosted Zones (hybrid DNS resolution)
 - └─ IPAM for centralized CIDR management
- └─ Security Baseline
 - └─ GuardDuty (org-wide, delegated admin in Security account)
 - └─ SecurityHub (CIS benchmark, PCI-DSS if needed)
 - └─ AWS Config rules (50+ compliance checks)
 - └─ CloudTrail: org-level trail → Log Archive account S3 bucket
 - └─ KMS: customer-managed keys per workload account
- └─ IaC Foundation
 - └─ Terraform modules: networking, security, eks, database, observability
 - └─ CI/CD: GitHub Actions + Terraform plan on PR, apply on merge
 - └─ State management: S3 + DynamoDB (per-account backends)

Phase 2 (Month 2-4): MIGRATE – "Wave-based execution"

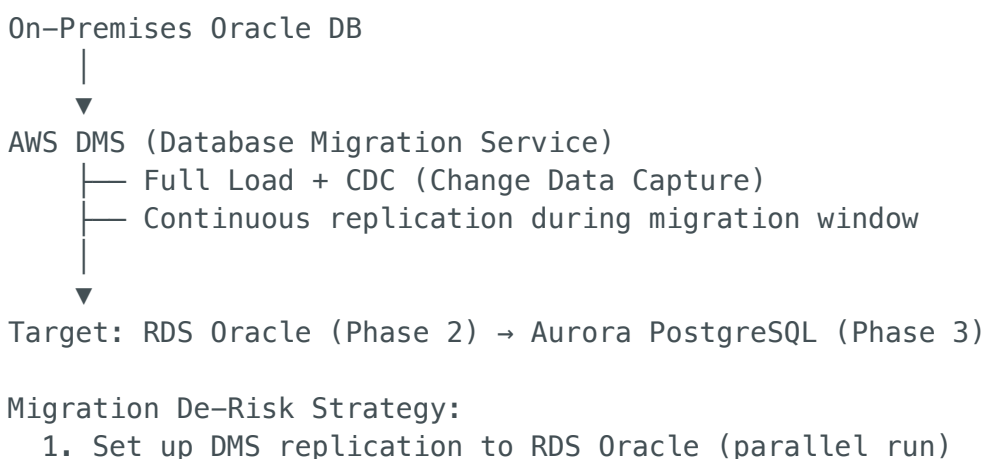
- └─ Discovery: AWS Application Discovery Service + Migration Hub
- └─ Wave Planning: Group by dependency (database first, then app tier)
- └─ Migration Execution:

- VMs → EC2 via AWS MGN (Application Migration Service)
 - Continuous block-level replication (agent-based)
 - Test instances in Migration-Staging account → validate
 - Cutover window: DNS switch → typically <1 hour downtime
- Oracle Database (TRADE-OFF DECISION – draw both options):
 - Option A: Amazon RDS for Oracle (lowest risk, same engine)
 - Pro: No application changes, fastest migration
 - Con: Oracle licensing cost continues (\$\$\$)
 - Tool: AWS DMS (Full Load + CDC for minimal downtime)
 - Option B: Aurora PostgreSQL (recommended for cost savings)
 - Pro: 3x performance, eliminates Oracle license (\$150K+/yr savings)
 - Con: Requires Schema Conversion Tool + app testing (add 4–6 weeks)
 - Tool: AWS SCT + DMS
- "I'd recommend: Replatform to RDS Oracle now to hit the timeline, then plan a Phase 3 migration to Aurora PostgreSQL as a separate modernization workstream. This de-risks the datacenter exit."
- File shares → Amazon FSx for Windows File Server
- Validation: Smoke tests per wave, parallel run period

Phase 3 (Month 4–6): OPTIMIZE & MODERNIZE ROADMAP

- Right-size EC2 instances (AWS Compute Optimizer recommendations)
- Savings Plans: 1-year Compute SP for stable workloads (30–40% savings)
- Observability: CloudWatch dashboards, X-Ray tracing, centralized logging
- Security hardening: Inspector scans, Macie for PII detection
- Day 2 Automation (Cornerstone 4):
 - EventBridge → Lambda: auto-remediate Config rule violations
 - Systems Manager Patch Manager: automated OS patching
 - Backup: AWS Backup with cross-region copy rules
 - Cost anomaly detection → SNS → Slack alerts
- Modernization Roadmap (hand off to client):
 - Candidate 1: .NET services → containerize on ECS Fargate
 - Candidate 2: Batch jobs → Lambda + EventBridge (serverless)
 - Candidate 3: Oracle → Aurora PostgreSQL (when ready)
 - "This roadmap ensures the client continues extracting value from AWS after the consulting engagement ends."

Step 5: Data Migration Detail (draw this clearly)



2. Run application test suite against RDS Oracle
3. Gradually shift read traffic (Route 53 weighted routing)
4. Cutover writes during maintenance window (<1 hr downtime)
5. Keep DMS reverse replication active for 2 weeks (rollback safety net)

Key Talking Points (say these out loud)

- "I'm recommending a **rehost-first strategy** to de-risk the datacenter exit and meet the 6-month deadline. But I want to be clear — this isn't the end state. The modernization roadmap is what delivers the long-term ROI."
- "For the Oracle database, I'd present the client with a **TCO analysis**: RDS Oracle licensing vs Aurora PostgreSQL migration effort. In my experience, the break-even on Oracle → PostgreSQL migration effort is typically 12-18 months of license savings."
- "Notice I've structured this in a multi-account landing zone from day one — this isn't optional for an enterprise migration. Single-account architectures create security, blast radius, and compliance problems that are expensive to fix later."
- "The Day 2 automation — Config auto-remediation, automated patching, cost alerting — is what separates a consulting engagement that delivers lasting value from one that leaves the client with a beautiful architecture they can't operate."

6. Scenario 2: Serverless-First Architecture on AWS {#6-scenario-2-serverless-first-architecture}

Prompt: "Design a serverless API backend for a fintech client that processes 10,000 transactions per minute with sub-100ms latency."

Discovery Questions (Cornerstone 1)

"For a fintech workload, I need to clarify several things:

Compliance: PCI-DSS? This affects VPC placement, encryption, and audit logging.

Latency: Is 100ms P99 or P50? This changes whether we need provisioned concurrency.

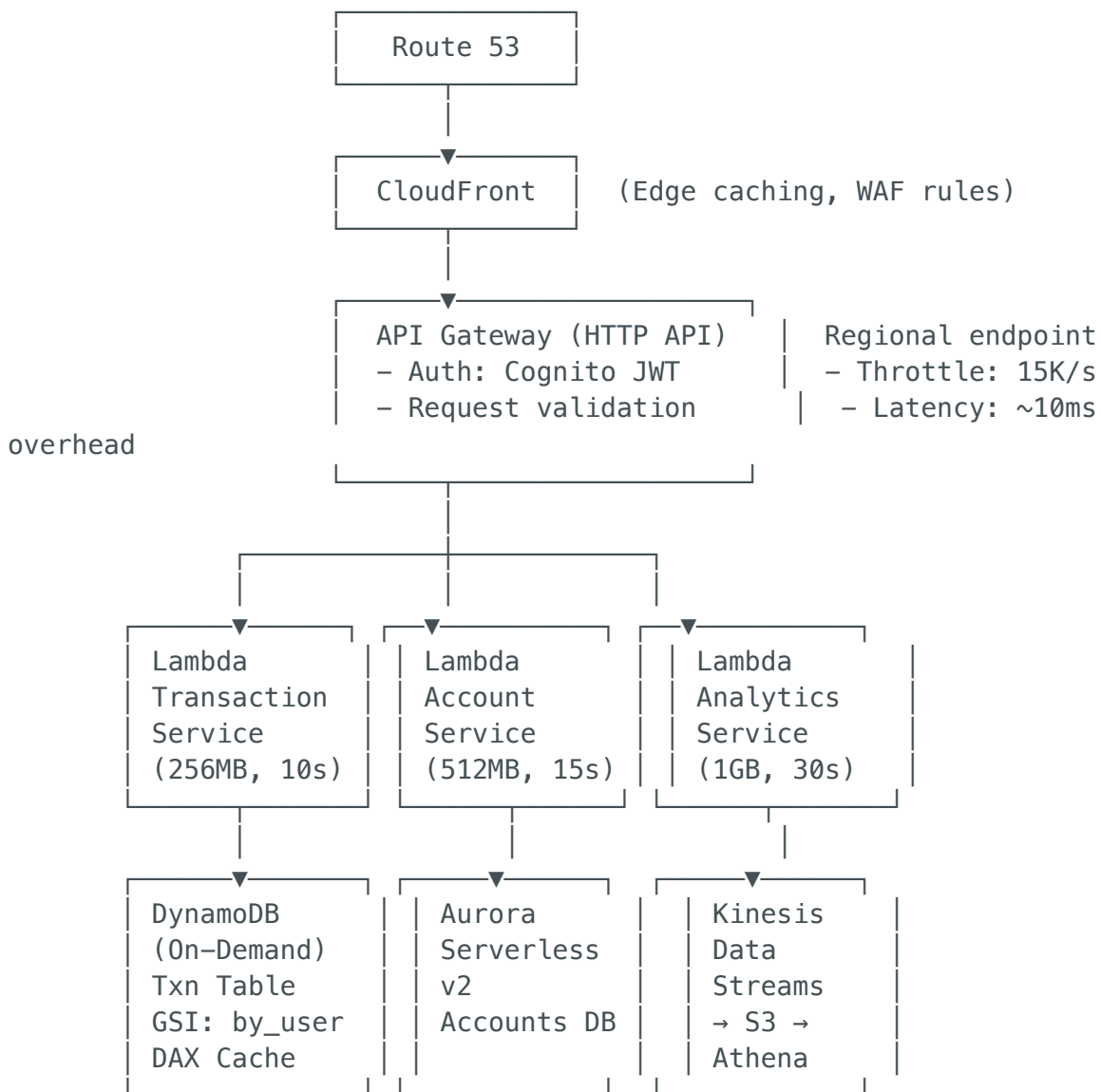
Traffic pattern: Steady 10K TPS or bursty? On-demand vs provisioned DynamoDB.

Data model: Relational joins needed, or key-value access patterns?

Existing stack: Are there legacy systems this API integrates with?

Team: Do they have Lambda/serverless experience, or is this new?"

Architecture (draw left-to-right)



Account Structure: This sits in the Workload OU → Production Account

- API Gateway, Lambda, DynamoDB all in Production account
- Logs flow to Log Archive account (CloudTrail + CloudWatch)
- CI/CD runs in Shared Services account (cross-account deploy role)

Key Design Decisions (Trade-Off Table)

Decision	Choice	Why — Trade-Off Articulation
API Gateway type	HTTP API (not REST API)	"HTTP API is 60% cheaper ($1/M$ vs $3.50/M$) and has lower latency (~10ms vs ~29ms). We lose API caching and usage plans, but for this workload the latency saving is more critical. If they needed per-client rate limiting, I'd switch to REST API."
Database for transactions	DynamoDB (not Aurora)	"Transactions are write-heavy with key-value access patterns — DynamoDB gives single-digit ms latency at any scale. I'm NOT choosing Aurora here because the relational overhead would add ~15ms latency. If the data model required complex joins, I'd reconsider."
Caching	DAX (DynamoDB Accelerator)	"Microsecond latency for hot reads, transparent to the application code. The alternative would be ElastiCache, but DAX requires zero code changes."
Cold start mitigation	Provisioned Concurrency = 50	"Fintech can't tolerate cold starts — we keep 50 warm instances for baseline traffic. This costs ~\$150/month but eliminates the P99 tail latency problem."
Async processing	Kinesis → Lambda → S3	"Transaction analytics processed asynchronously — doesn't block the API. I chose Kinesis over SQS because we need ordering guarantees and replay capability for financial data."

Performance Budget (draw this)

Total latency budget: 100ms (P99)	
API Gateway overhead:	~8ms
Lambda cold start:	0ms (Provisioned Concurrency)
Lambda execution:	~20ms
DynamoDB write:	~10ms (single-digit ms)
DAX read:	~1ms (microsecond)

Network overhead: ~5ms

Total: ~44ms (well within 100ms budget, 56ms headroom)

Day 2 Operations (Cornerstone 4)

Observability:

- Lambda: CloudWatch Metrics (invocations, errors, duration, throttles)
- DynamoDB: ConsumedReadCapacity, ThrottledRequests,

SuccessfulRequestLatency

- Custom metric: transactions_processed_per_second (business KPI)
- X-Ray: end-to-end trace from API Gateway → Lambda → DynamoDB

Auto-Remediation:

- DynamoDB throttle alarm → EventBridge → Lambda → switch to provisioned capacity and scale up (for traffic spikes beyond on-demand burst)
- Lambda error rate > 5% → EventBridge → SNS → PagerDuty (page on-call)
- API Gateway 5xx > 1% → auto-rollback via CodeDeploy traffic shifting

IaC: All resources defined in Terraform with SAM/CDK for Lambda packaging

Cost Estimate

10,000 TPS × 60 seconds = 600K requests/minute

Lambda: \$0.20/1M requests + compute time

= ~25.9B requests/month → ~\$5,180 (requests) + ~\$3,000 (compute)

DynamoDB (On-Demand): ~\$10,000/month

Total estimated: ~\$18K–20K/month

Compared to EC2-based: ~\$25–30K/month (including HA, ops overhead)

Savings: ~30% + zero patching/scaling ops overhead

7. Scenario 3: Multi-Account Landing Zone Design {#7-scenario-3-multi-account-landing-zone}

Prompt: "Design an AWS multi-account landing zone for an enterprise with 200 developers, 50 applications, and strict compliance requirements."

Discovery Questions

"This is a foundational engagement – the kind Caylent excels at. Let me clarify:

Compliance: Which frameworks? SOC2, PCI-DSS, HIPAA, FedRAMP? Each adds specific controls.

Geography: Which AWS regions? EU data residency requirements?

Existing state: Greenfield or brownfield? Existing AWS accounts to bring under management?

Network: On-prem connectivity needed? How many physical locations?

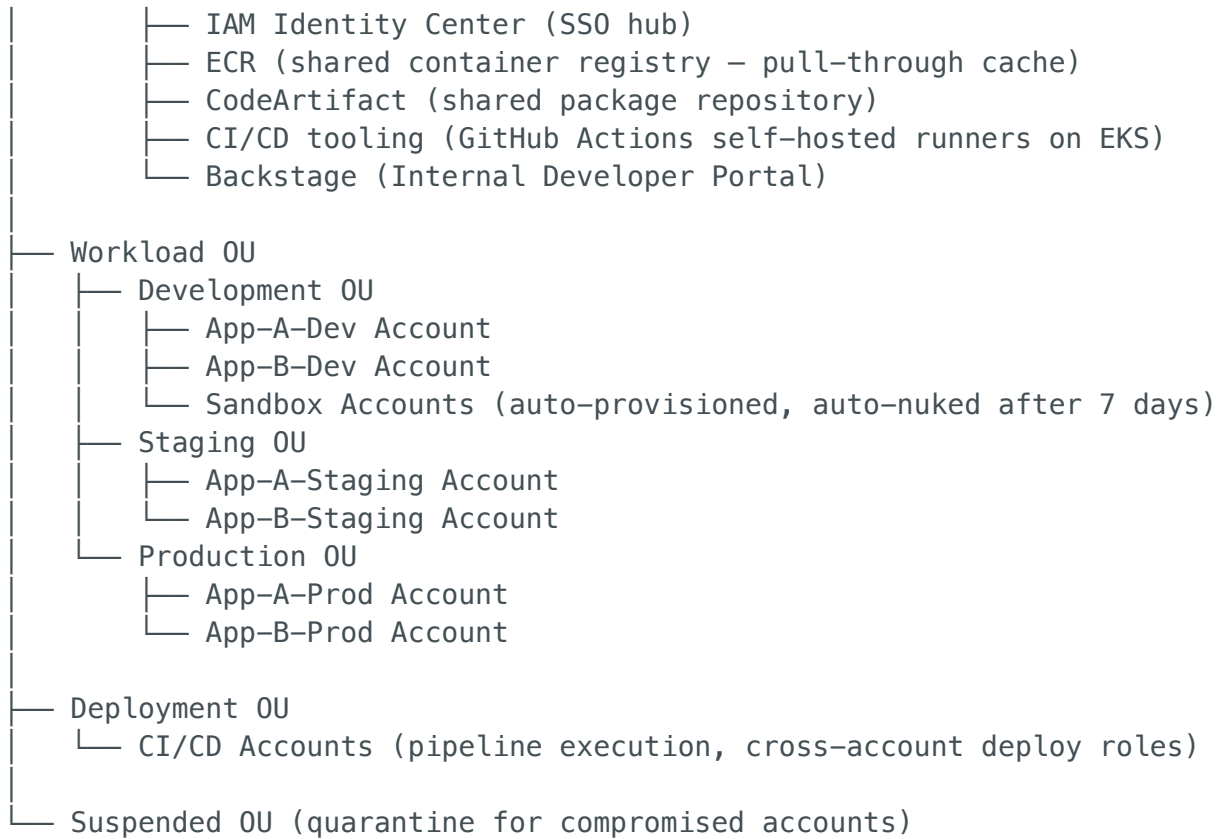
Identity: Existing IdP? Okta, Azure AD, on-prem AD?

Team model: Platform team managing centrally, or federated teams per app?"

Architecture (this is Cornerstone 2 in full)

AWS Organization Root

- Management Account (billing, org policies, SS0)
 - ★ Minimal workloads here – only billing and org management
- Security OU
 - Log Archive Account
 - CloudTrail (org trail), Config, VPC Flow Logs, S3 access logs
 - S3 bucket with Object Lock (WORM – compliance requirement)
 - Security Audit Account
 - GuardDuty (delegated admin), SecurityHub, Inspector
 - Detective (entity graphs for investigation)
 - Security Tools Account
 - WAF centralized (Firewall Manager), Network Firewall rules
- Infrastructure OU
 - Network Account (Hub)
 - Transit Gateway (hub-and-spoke, route tables per OU)
 - Direct Connect Gateway (if on-prem connectivity)
 - Network Firewall (inspection VPC – all traffic inspected)
 - Route 53 Private Hosted Zones (centralized DNS)
 - AWS IPAM (IP address management across all accounts)
 - VPN endpoints (backup connectivity)
 - Shared Services Account



Service Control Policies (SCPs) — Draw Key Ones

```
// SCP 1: Deny actions outside approved regions
{
  "Statement": [{
    "Sid": "DenyOutsideApprovedRegions",
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-west-2", "eu-west-1"]
      },
      "ArnNotLike": {
        "aws:PrincipalARN": "arn:aws:iam::*:role/OrganizationAdmin"
      }
    }
  }]
}

// SCP 2: Prevent disabling security services
{
  "Statement": [{
    "Sid": "PreventSecurityDisable",
    "Effect": "Deny",
    "Action": [
      "guardduty:DeleteDetector",
```



```

    "config:StopConfigurationRecorder",
    "cloudtrail:StopLogging",
    "cloudtrail:DeleteTrail"
  ],
  "Resource": "*"
}]
}

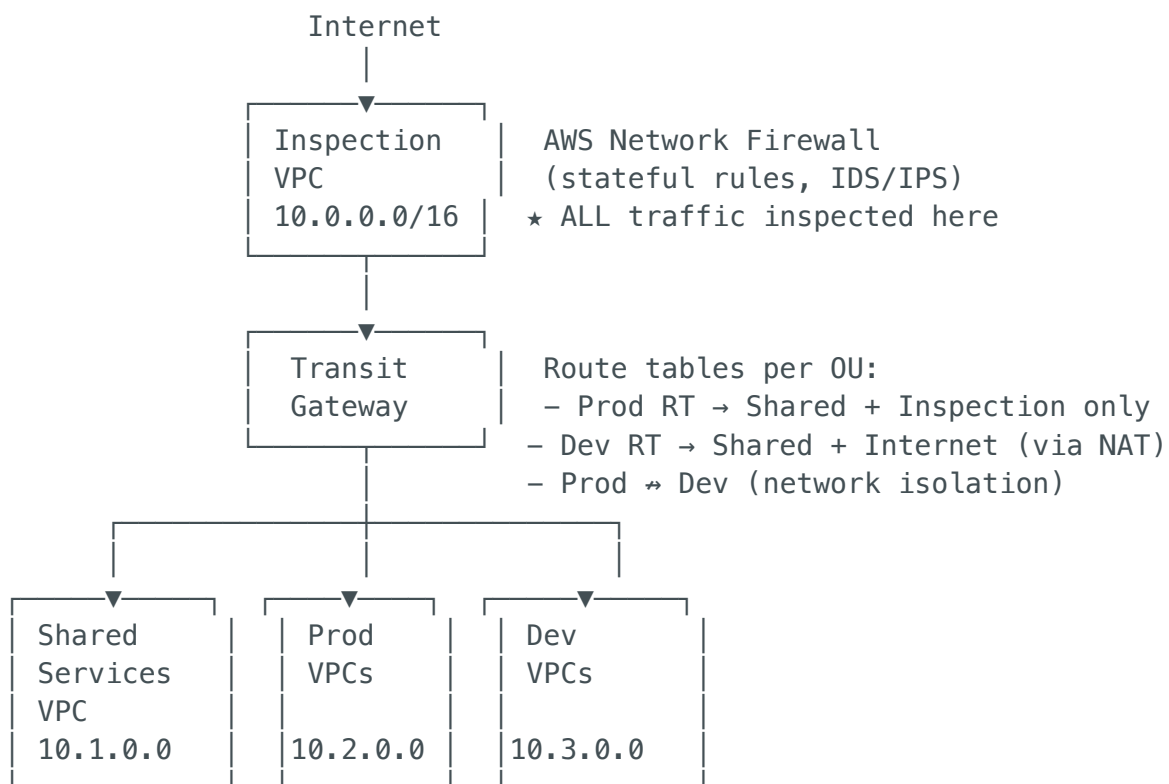
```

```

// SCP 3: Enforce encryption (S3, EBS)
// SCP 4: Deny public S3 buckets
// SCP 5: Require IMDSv2 for EC2

```

Network Architecture (Hub-and-Spoke via Transit Gateway)



Account Vending Machine (Day 2 Operations)

Developer requests new account via Backstage / ServiceNow



Account Factory for Terraform (AFT) – OR Step Functions workflow:

1. Validate request (approved manager, budget code, OU assignment)
2. Create AWS Account via Organizations API
3. Move to correct OU (Dev/Staging/Prod)

4. Apply baseline via Terraform:
 - VPC with standard CIDR from IPAM
 - TGW attachment to Network Hub
 - CloudTrail, Config, GuardDuty enabled
 - Default IAM roles (Admin, Developer, ReadOnly)
 - Tagging enforcement (Config rules)
 - VPC endpoints (S3, ECR, STS)
5. Create DNS delegation (Route 53)
6. Configure SSO permission sets
7. Notify requestor via Slack



Account ready in ~15 minutes (fully automated, fully governed)

"This is the IaC-first approach that Caylent delivers – every account is consistent, compliant, and auditable from day one."

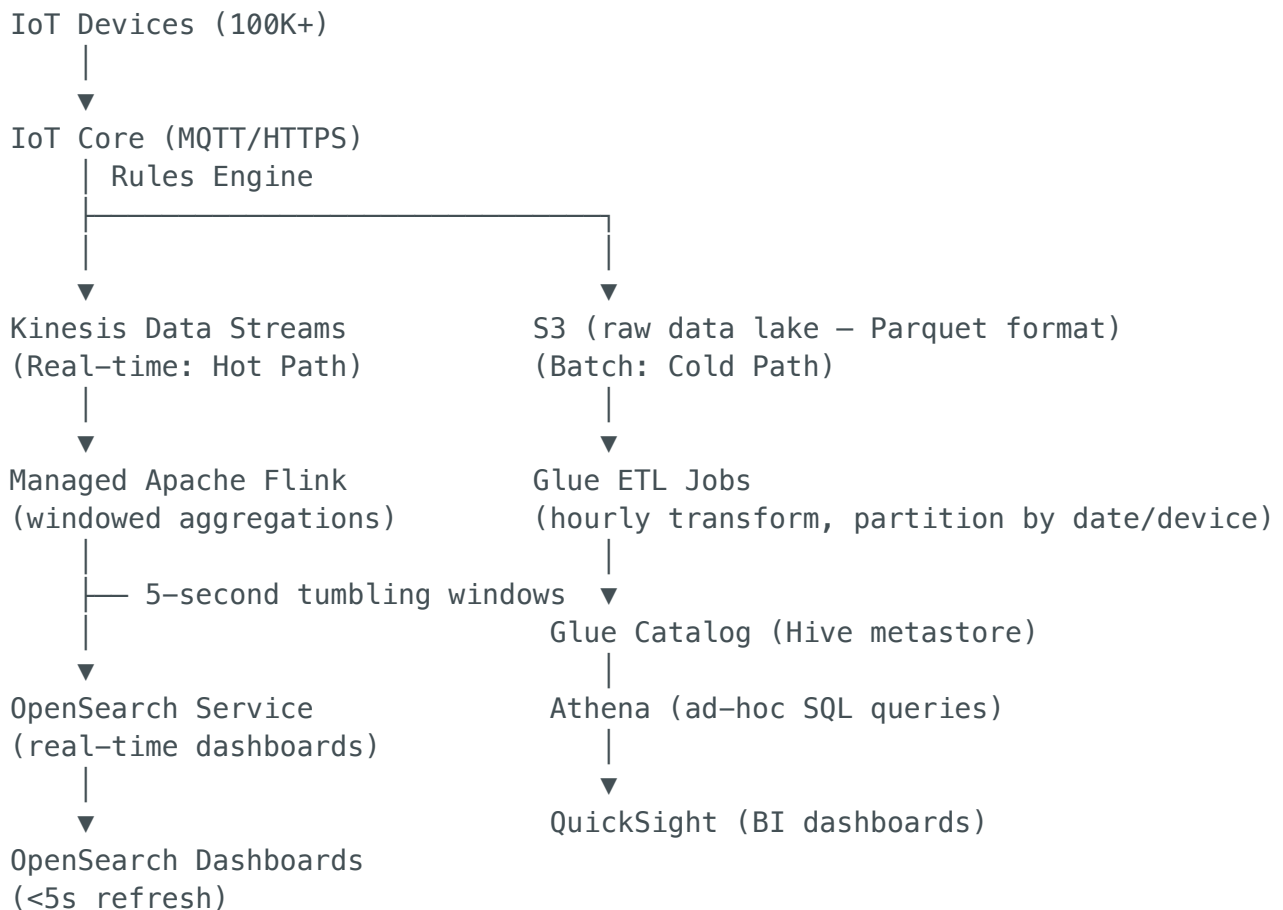
8. Scenario 4: Real-Time Data Platform Architecture {#8-scenario-4-real-time-data-platform}

Prompt: "A client needs a real-time data analytics platform. They have 500GB of streaming data per day from IoT devices and need dashboards with <5 second refresh."

Discovery Questions

"What kind of IoT devices? Protocol – MQTT, HTTPS, or custom?
How many devices? 10K vs 1M changes the ingestion layer.
What analytics – simple aggregations or complex ML inference?
Who consumes dashboards – executives, ops teams, automated systems?
Data retention – how long do they need hot data vs cold archive?
Compliance – is this industrial data, health data, or consumer data?"

Architecture (Lambda Architecture – Hot + Cold Paths)



Key Design Decisions

Decision	Choice	Trade-Off Rationale
Ingestion	IoT Core + Kinesis Streams	"IoT Core handles MQTT natively; Kinesis handles the scale at 1MB/s per shard. I considered MSK but the operational overhead isn't justified for a streaming pipeline that doesn't need Kafka's consumer group model."
Stream processing	Managed Apache Flink	"Managed Flink gives us exactly-once semantics and SQL for aggregations. The alternative is Kinesis Data Analytics, but Flink's windowing functions are more powerful for the 5-second aggregation requirement."

Decision	Choice	Trade-Off Rationale
Dashboard store	OpenSearch	"Sub-second query latency for time-series data. I chose this over Timestream because OpenSearch also gives us full-text log search capability — two-for-one value."
Shard calculation	500GB/day ÷ 86400s ÷ 1MB/s ≈ 6 shards + buffer = 10	"Right-sized for throughput with headroom for spikes."

Day 2 Operations

Auto-Remediation:

- Kinesis IteratorAge > 5 seconds → EventBridge → Lambda → increase shard count (UpdateShardCount API)
- OpenSearch cluster health RED → EventBridge → SNS → PagerDuty
- S3 bucket growing beyond budget threshold → EventBridge → Lambda → apply Intelligent-Tiering lifecycle policy

Observability:

- Custom CloudWatch dashboard: ingestion rate, processing lag, query latency
- Business KPI: "time from device event to dashboard update" (target: <5s)

9. Scenario 5: Modernization — Monolith to Microservices on EKS {#9-scenario-5-monolith-to-microservices}

Prompt: "A client has a Java monolith serving 5M users. They want to modernize to microservices on EKS. Design the modernization strategy and target architecture."

Discovery Questions

"How big is the monolith? LOC, number of modules, team size?
What's the deployment frequency today? This tells me the pain level.
Has the team worked with containers or Kubernetes before?
Are there database coupling issues – single shared DB?
What's the business driver – scaling, speed of feature delivery, or both?"

Strangler Fig Strategy (Cornerstone 3 – Modernization)

TRADE-OFF DECISION (draw this first):

Option A: Big Bang Rewrite

Risk: HIGH (most rewrites fail)

Timeline: 12–18 months

Business value during rewrite: ZERO

"I would NOT recommend this. In 22 years, I've seen one successful big-bang rewrite. The rest were cancelled or over-budget."

Option B: Strangler Fig (RECOMMENDED)

Risk: LOW (incremental, reversible)

Timeline: 9–12 months

Business value: Continuous (new features can ship during migration)

"This is the proven pattern for monolith decomposition."

Phase 1: Foundation (Month 1–2)

- └─ Set up EKS cluster (private, multi-AZ, Karpenter for autoscaling)
- └─ Set up CI/CD (GitHub Actions CI → ArgoCD GitOps CD)
- └─ Set up observability (ADOT → AMP → Managed Grafana)
- └─ Deploy monolith as a container on EKS (no code changes yet)
 - └─ This proves: container builds, deployment pipeline, monitoring
 - └─ "Early win – the monolith runs on EKS Day 1. Team gains K8s confidence."
- └─ IaC: Entire EKS cluster + add-ons defined in Terraform modules

Phase 2: Strangler Fig (Month 3–8)

- └─ Domain-Driven Design: identify bounded contexts
 - └─ User Service (auth, profile)
 - └─ Product Service (catalog, search)
 - └─ Order Service (cart, checkout)
 - └─ Payment Service (transactions)
 - └─ Notification Service (email, SMS, push)
 - └─ Analytics Service (events, reporting)
- └─ Extract ONE service at a time:
 1. Start with lowest-risk bounded context (e.g., Notifications)
 - └─ "I always start with the least coupled service – it de-risks

- the pattern before we tackle the harder ones."
- 2. Build as new microservice on EKS
- 3. Route traffic via API Gateway / Istio VirtualService:
 - /notifications → new microservice
 - Everything else → monolith
- 4. Canary traffic: 5% → 25% → 100% (Argo Rollouts)
- 5. Remove code from monolith
- 6. Repeat for next service

Phase 3: Full Microservices (Month 9–12)

- └ All services extracted, monolith decommissioned
- └ Event-driven communication (EventBridge + SQS)
- └ Database per service (each service owns its data)

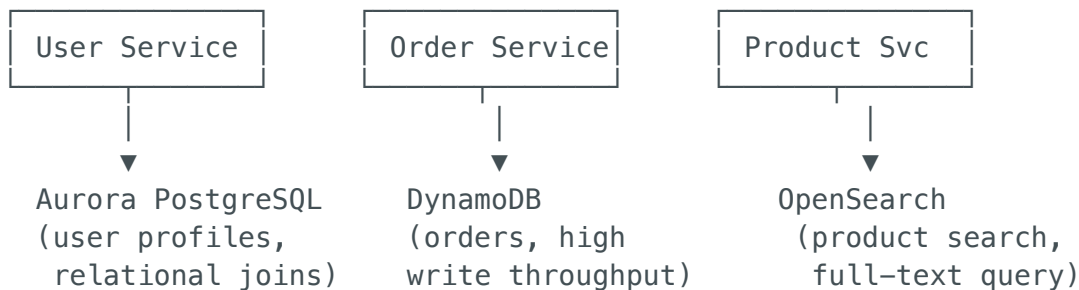
Target EKS Architecture

EKS Cluster (Private endpoint, Multi-AZ, v1.30+)

- └ System Namespace (kube-system)
 - └ CoreDNS, kube-proxy, VPC-CNI (prefix delegation for IP density)
 - └ Karpenter (node autoscaling – "I mention Karpenter specifically because it's more efficient than Cluster Autoscaler: it provisions right-sized nodes in seconds, not minutes")
 - └ AWS Load Balancer Controller
 - └ External Secrets Operator → Secrets Manager
 - └ cert-manager → ACM
 - └ Kyverno (policy engine – require labels, block privileged, enforce limits)
- └ ArgoCD Namespace
 - └ ArgoCD (GitOps – pull-based deployment)
 - └ ApplicationSets (auto-generate apps per team)
 - └ Argo Rollouts (canary + analysis – error rate < 1%, P99 < 500ms)
- └ Istio System Namespace
 - └ istiod (control plane)
 - └ Istio Ingress Gateway
 - └ "Service mesh gives us mTLS (zero-trust between pods), traffic management, and observability – three things the client needs but shouldn't build themselves."
- └ App Namespace: user-service
 - └ Deployment (3 replicas, HPA: CPU 70%)
 - └ IRSA (pod-level IAM – "no shared IAM roles, least privilege per pod")
 - └ PeerAuthentication (STRICT mTLS)
 - └ AuthorizationPolicy (allow from: order-service, api-gateway)
- └ App Namespace: order-service
 - └ Deployment (5 replicas, HPA + KEDA: scale on SQS queue depth)
 - └ SQS consumer (event-driven scaling)

- └─ Observability Namespace
 - └─ ADOT Collector → AMP (metrics) + X-Ray (traces)
 - └─ Fluent Bit → CloudWatch Logs (structured JSON, correlation ID)
 - └─ Grafana dashboards (per-service RED metrics)
- └─ Node Strategy
 - └─ System: m7i.xlarge (on-demand, 3 nodes – always available)
 - └─ Application: Karpenter provisioner (spot + on-demand mix, Graviton)
 - "Using Graviton ARM instances saves 20% cost with 40% better price-performance. Karpenter handles the bin-packing automatically."
 - └─ GPU: g5.xlarge (if ML inference pods needed)

Database Per Service



TRADE-OFF: "I'm choosing different databases per service based on their access patterns – not just defaulting to one database. Aurora for relational needs, DynamoDB for high-throughput writes, OpenSearch for search. This is the polyglot persistence pattern."

Communication: EventBridge (async events between services)

- UserCreated → Order Service (create customer record)
- OrderPlaced → Notification Service (send email)
- ProductUpdated → Search Service (re-index in OpenSearch)

10. Scenario 6: DR & High Availability Architecture {#10-scenario-6-dr-and-high-availability}

Prompt: "Design a DR strategy for a banking client with RPO < 1 minute and RTO < 15"

minutes."

Discovery Questions

"RPO < 1 minute and RTO < 15 minutes – that's a Hot Standby or Warm Standby DR tier. Let me clarify:

Active-Active or Active-Passive? This changes cost significantly.

Regulatory: Is this a banking regulator requirement (OCC, FDIC)?

Current DR: What's in place today? Tape backup? Async replication?

Failover trigger: Automated or manual approval required?

Testing: How often do you need to drill? Quarterly? Monthly?"

Multi-Region Architecture

Primary Region: us-east-1

DR Region: us-west-2

Route 53 (Primary)
Health check: 10s, 3/3

CloudFront + WAF

ALB (active)

EKS Cluster (full)
– 5 app replicas
– Full Karpenter nodes

Aurora Global DB
(Writer Instance)
RPO: < 1 second

ElastiCache (Global)

S3 (Cross-Region Repl.)

Route 53 (Failover)
60s TTL for fast switch

CloudFront + WAF

ALB (standby, min cap)

EKS Cluster (pilot)
– 1 replica (warm)
– Karpenter ready

Aurora Read Replica
(Promoted on failover)
Async repl. lag < 1s

ElastiCache (Replica)

S3 (Replica bucket)

Failover Sequence (< 15 min RTO):

1. CloudWatch alarm detects primary unhealthy

(2 min)

2. EventBridge → Step Functions failover automation

(1 min)

3. Aurora Global DB: promote secondary to writer

(< 1 min)

4. EKS: Karpenter scales up DR cluster

(2–3 min)

5. Route 53 health check fails → DNS failover

(60s TTL)

6. ElastiCache Global Datastore: promote replica

(< 1 min)

7. Automated smoke tests validate DR region

(2 min)

Total: ~10–12 minutes

TRADE-OFF: "Active-passive gives us < 15 min RTO at ~40% of the cost of active-active. For banking, this meets regulatory requirements without doubling infrastructure spend. If the RTO requirement were < 1 minute, I'd recommend active-active with DynamoDB Global Tables."

RTO/RPO Analysis

Component	RPO	RTO	AWS Mechanism
Database	< 1 second	< 1 minute	Aurora Global DB (async repl. lag < 1s)
Cache	< 1 second	< 1 minute	ElastiCache Global Datastore
File Storage	< 15 minutes	< 5 minutes	S3 Cross-Region Replication
Application	N/A (stateless)	< 5 minutes	EKS + Karpenter auto-scale
DNS	N/A	< 2 minutes	Route 53 failover (60s TTL)
Secrets	< 1 minute	< 1 minute	Secrets Manager multi-region

Day 2: DR Drill Automation

```
# Quarterly DR drill – fully automated via Step Functions
# "At Caylent, we'd build this as part of the engagement deliverable,
# not leave it as a manual runbook."
```

DR Drill Workflow (Step Functions):

1. Simulate failure: disable Route 53 health check for primary
2. Monitor: wait for automated failover (timer)
3. Validate: run full test suite against DR endpoint
4. Measure: actual RTO and RPO vs targets
5. Generate: compliance report (PDF → S3 → email to CISO)
6. Failback: re-enable primary, verify replication catches up
7. Post-drill: Slack notification with results + dashboard link

Auto-Remediation During Failover:

EventBridge → Lambda:

- Scale up DR EKS nodes
- Update external DNS entries
- Invalidate CloudFront cache
- Notify ops team via PagerDuty
- Create incident ticket in ServiceNow

11. Scenario 7: CI/CD & Platform Engineering {#11-scenario-7-cicd-and-platform-engineering}

Prompt: "Design a CI/CD platform for 50 development teams deploying to EKS, with security gates and compliance controls."

Discovery Questions

"50 teams is a platform engineering problem, not just a CI/CD problem."

Languages: Polyglot (Java, Python, Node, Go)? This affects build tooling.

Compliance: Do they need SBOM, image signing, CVE gates? (SOC2/PCI?)

Deployment model: GitOps or traditional push? What's their Git hosting?

Current state: Are they deploying with scripts today, or have some automation?

Tenancy: Single EKS cluster or cluster-per-team?"

Platform Architecture (Cornerstone 4 — this IS Day 2 Operations)

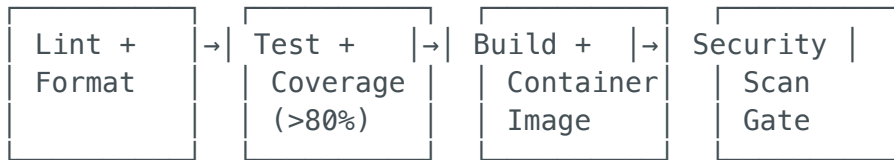
DEVELOPER EXPERIENCE LAYER

- Backstage (Internal Developer Portal)
 - Service Catalog: create new service from golden templates
 - TechDocs: auto-generated from repo (no stale wikis)
 - Scorecards: security, compliance, quality metrics per team
- Golden Templates (Cookiecutter / Scaffolding)
 - Java Spring Boot microservice template
 - Python FastAPI template
 - Terraform module template
 - Each includes: CI pipeline, Dockerfile, Helm chart, observability
 - "Golden templates encode Caylent's best practices – every new service starts with security, CI/CD, and observability baked in."

CI/CD PIPELINE LAYER

- GitHub Actions (CI)
 - Shared workflows (org-level reusable actions)

- Self-hosted runners on EKS (for VPC access, cost savings)
- Pipeline stages (draw these as sequential gates):



Security Gate Details:

- Trivy (CVE scan – CRITICAL = pipeline fails)
- Checkov (IaC scan – HIGH = pipeline fails)
- TruffleHog (secrets detection)
- SBOM generation (Syft → Dependency-Track)
- Image signing (cosign – signature verified at deploy)

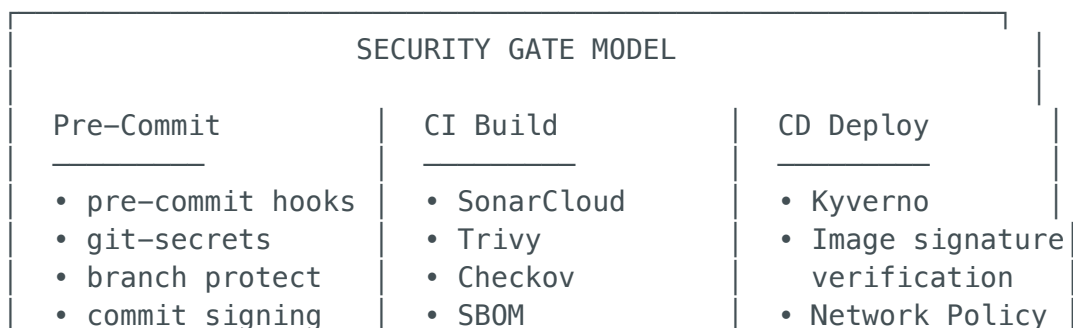
ArgoCD (CD – GitOps)

- ApplicationSets: auto-generate apps per team/env
- Sync Policies:
 - Dev: auto-sync (deploy on merge to dev branch)
 - Staging: auto-sync + auto-prune
 - Prod: manual sync (requires approval in ArgoCD UI)
- Progressive Delivery (Argo Rollouts):
 - Canary: 5% → 25% → 50% → 100%
 - Analysis template: error rate < 1%, P99 < 500ms
 - Auto-rollback on analysis failure
- RBAC: team can only deploy to their namespace

PLATFORM SERVICES LAYER (what platform team provides)

- Crossplane: Kubernetes-native IaC for AWS resources
 - "Developers request RDS, S3, SQS via Kubernetes manifests – the platform team controls WHAT they can request via XRDs."
- External Secrets Operator → Secrets Manager
- cert-manager → ACM
- Kyverno Policies:
 - Require resource requests/limits
 - Block privileged containers
 - Require approved image registries (ECR only)
 - Require labels: team, cost-center, tier
 - Block latest tag (must use SHA or semver)
- KEDA (event-driven autoscaling – SQS, Kafka, etc.)

Security Gate Model



- | | |
|---|---|
| <ul style="list-style-type: none"> • License check • cosign signing | <ul style="list-style-type: none"> validation • OPA/Gatekeeper • Manual approval (prod only) |
|---|---|

"Three gates = defense in depth for the software supply chain. If code passes all three, we have high confidence in what's running in production."

12. Scenario 8: Security Architecture & Compliance {#12-scenario-8-security-architecture}

Prompt: "Design a security architecture for a healthcare client on AWS that must comply with HIPAA."

Discovery Questions

"HIPAA changes the architecture fundamentally. Let me clarify:

PHI scope: Which systems handle Protected Health Information?

BAA: Have they signed the AWS Business Associate Agreement?

HIPAA-eligible services: Only certain AWS services are covered.

Existing controls: What compliance tooling is in place today?

Audit: Are they expecting a HITRUST certification or self-attestation?"

Defense-in-Depth Architecture (6 Layers)

Layer 1: Edge Security

- CloudFront + WAF (OWASP Top 10, rate limiting, geo-blocking)
- Shield Advanced (DDoS protection with 24/7 response team)
- Route 53 DNSSEC (DNS integrity)

Layer 2: Network Security

- VPC: private subnets ONLY (no public-facing instances)

- Security Groups (stateful, reference SG IDs not CIDRs)
- Network Firewall (stateful inspection, IDS/IPS – Inspection VPC)
- VPC Flow Logs → Log Archive Account → Athena (network forensics)
- VPC Endpoints: S3, DynamoDB, ECR, STS, KMS (no internet needed)

Layer 3: Identity & Access

- IAM Identity Center (SSO + MFA enforced – no exceptions)
- IAM roles with least privilege (no long-lived access keys)
- IRSA for EKS pods (pod-level AWS permissions)
- SCPs at org level (prevent security service disabling)
- IAM Access Analyzer (continuous analysis of unused permissions)

Layer 4: Data Protection

- Encryption at rest: KMS CMK (customer-managed keys)
 - S3: SSE-KMS (with bucket key for cost optimization)
 - RDS/Aurora: KMS encryption (transparent to application)
 - EBS: KMS encryption (all volumes)
 - DynamoDB: KMS encryption
- Encryption in transit: TLS 1.2+ everywhere
 - ACM certificates (auto-renewal)
 - Istio mTLS (pod-to-pod – zero trust)
 - RDS: require_ssl = true (connection policy)
- Key rotation: automatic annual rotation (KMS)
- PHI detection: Macie (scan S3 for PHI patterns)

Layer 5: Detection & Response

- GuardDuty (threat detection – crypto mining, credential abuse)
- SecurityHub (unified findings, CIS + HIPAA benchmark checks)
- Config Rules (50+ compliance checks – continuous monitoring)
- CloudTrail (org-level trail, all regions, all accounts)
- Inspector (EC2/ECR vulnerability scanning)
- Detective (security investigation, entity relationship graphs)

Layer 6: Automated Incident Response (Cornerstone 4 – AI-Driven Ops)

- EventBridge rules for automated response:
 - GuardDuty High Severity → Lambda → isolate instance (remove all SGs, add quarantine SG, snapshot EBS, page SecOps)
 - Config non-compliant → Lambda → auto-remediate (e.g., unencrypted S3 bucket → enable KMS encryption)
 - CloudTrail root login → SNS → PagerDuty IMMEDIATELY
- Systems Manager Incident Manager (runbooks, escalation)
- "These automated response patterns are what Caylent builds as reusable modules – every healthcare client gets them out of the box."

HIPAA-Specific Controls

HIPAA Requirement	AWS Implementation	Caylent Delivery Detail
Access controls (§164.312(a))	IAM Identity Center, MFA, RBAC, IRSA	Terraform module: identity/

HIPAA Requirement	AWS Implementation	Caylent Delivery Detail
Audit controls (§164.312(b))	CloudTrail, VPC Flow Logs, S3 access logs	All logs → Log Archive (immutable)
Integrity (§164.312(c))	KMS encryption, S3 Object Lock, versioning	Config rule: enforce encryption
Transmission security (§164.312(e))	TLS 1.2+, VPN/Direct Connect, mTLS	Network Firewall TLS inspection
PHI protection	Macie detection, S3 bucket policies, DLP	Weekly Macie scan, auto-quarantine
BAA	Sign AWS BAA, use ONLY HIPAA-eligible services	SCP: deny non-eligible services
Breach notification	GuardDuty → EventBridge → SNS → SecOps	Automated incident workflow

13. Scenario 9: Cost Optimization & FinOps {#13-scenario-9-cost-optimization}

Prompt: "A client's AWS bill jumped from 200K to 500K/month. How would you investigate and optimize?"

Investigation Framework

Step 1: Identify Cost Spike Source (Day 1)

— Cost Explorer: filter by service, account, tag, linked account

— Cost Anomaly Detection: did it fire? If not, set it up immediately

— Common culprits (check these first):

— NAT Gateway data transfer (hidden cost #1 in AWS)

— EC2: untagged instances left running in dev/sandbox accounts

— RDS: over-provisioned Multi-AZ in non-prod environments

- └─ S3: no lifecycle policies, data growing unchecked
- └─ Lambda: recursive invocation, memory over-provisioned
- └─ Data transfer: cross-AZ, cross-region, internet egress

Step 2: Right-Sizing Analysis (Week 1-2)

- └─ Compute Optimizer: EC2, Lambda, EBS, ECS recommendations
- └─ CloudWatch metrics: instances with CPU avg < 20% = over-provisioned
- └─ Trusted Advisor: idle resources, low-utilization instances
- └─ Custom queries:
 - RDS instances with < 10% CPU average
 - EBS volumes with 0 IOPS (unattached)
 - Elastic IPs not attached to running instances

Step 3: Quick Wins (Month 1) – aim for 20-30% reduction

- └─ Delete unused resources: old AMIs, unattached EBS, idle ELBs
- └─ Right-size EC2: t3.xlarge → t3.medium where CPU < 20%
- └─ S3 lifecycle: Standard → IA at 30d, Glacier at 90d, Deep Archive at 180d
- └─ Savings Plans: 1-year No Upfront Compute SP (stable workloads)
- └─ Graviton migration: 20% cost savings, 40% better perf
- └─ Schedule: stop dev/staging outside business hours (6pm-8am, weekends)
- └─ "This alone typically saves 60% on non-prod compute."
- └─ VPC Endpoints: S3, DynamoDB, ECR (eliminates NAT Gateway charges)

Step 4: Structural Optimization (Month 2-3)

- └─ NAT Gateway:
 - └─ VPC Endpoints for all AWS services (biggest cost saver)
 - └─ Consolidate VPCs (fewer NAT Gateways)
 - └─ VPC Flow Logs → Athena: find top talkers through NAT
- └─ Data transfer:
 - └─ CloudFront for S3 egress (cheaper than direct)
 - └─ Same-AZ placement where possible (free vs \$0.01/GB cross-AZ)
 - └─ Compress data in transit
- └─ Container optimization (Cornerstone 3 – modernize for cost):
 - └─ Karpenter consolidation (bin-pack pods, terminate underutilized nodes)
 - └─ Fargate Spot for non-critical workloads
 - └─ Spot instances for batch, dev/test (60-90% savings)
- └─ Serverless migration for variable workloads:
 - └─ EC2 cron jobs → Lambda + EventBridge (pay only when running)

FinOps Operating Model (draw this)

FinOps Operating Model

INFORM (Visibility)

- └─ Tagging strategy: Team, Project, Environment, CostCenter (enforced via Config rules)
- └─ Cost allocation: AWS Cost Categories
- └─ Dashboards: CUR → S3 → Athena → QuickSight
- └─ Monthly cost reviews per team (chargeback)

OPTIMIZE (Efficiency)

- Automated right-sizing recommendations
- RI/SP coverage target: 70%+ of steady-state
- Anomaly detection: > 20% daily spike → alert
- Spot adoption for eligible workloads

OPERATE (Governance)

- Budget alerts per account (80%, 100%, 120%)
- Automated shutdown: dev/staging off-hours
EventBridge schedule → Lambda → stop instances
- Quarterly commitment reviews
- Chargeback/showback to business units

"This FinOps model is something Caylent would implement as Terraform modules + Lambda functions – automated, not manual spreadsheets."

14. Scenario 10: Observability at Enterprise Scale {#14-scenario-10-observability}

Prompt: "Design an observability strategy for 50 microservices on EKS."

Three Pillars Architecture

OBSERVABILITY PLATFORM

METRICS

ADOT Collector
→ Amazon Managed Prometheus (AMP)
→ Managed Grafana

Key Metrics:

- RED (Rate, Error, Duration) per service
- USE (Utilization, Saturation, Errors) per node
- Business metrics

LOGS

Fluent Bit DaemonSet
→ CloudWatch Logs
→ Insights queries
→ Alerts on ERROR

Log Strategy:

- Structured JSON
- Correlation ID in every request
- 30-day hot (CW), 90-day cold (S3)
- NO PII in logs

TRACES

AWS X-Ray via ADOT Collector
→ X-Ray Console

Sampling:

- 5% normal

(orders/sec,
revenue/min)

- Log levels enforced via golden templates
- 100% error

ALERTING (SLO-Based – not raw thresholds)

SLO: 99.9% availability (error budget: 43.2 min/month)

Tier 1 (P1 – Page): SLO burn rate > 10x for 5 min

Tier 2 (P2 – Slack): SLO burn rate > 5x for 15 min

Tier 3 (P3 – Ticket): Degraded performance, non-critical

Alert Routing: PagerDuty → on-call (15 min ACK)

Escalation: 30 min → team lead → 60 min → VP Engineering

AI-DRIVEN AUTO-REMIEDIATION (Caylent emphasis):

- P1 alert fires → EventBridge → Lambda → auto-scale pods
- Pod crash loop → EventBridge → Lambda → rollback last ArgoCD sync (automated rollback)
- Node unhealthy → Karpenter auto-replaces (built-in)

Grafana Dashboard Hierarchy

Level 1: Executive Dashboard

- System health (green/yellow/red per service)
- SLO compliance percentage
- Error budget remaining (visual: "23 minutes of error budget left")
- Business KPIs (transactions/sec, revenue/min)

Level 2: Service Dashboard (per microservice – auto-generated from template)

- Request rate, error rate, latency (P50/P95/P99)
- Pod count, CPU, memory utilization
- Upstream/downstream dependency health (Istio metrics)

Level 3: Infrastructure Dashboard

- EKS node utilization, Karpenter scaling events
- Aurora connections, replication lag, IOPS
- ElastiCache hit rate, evictions, memory

Level 4: Debug Dashboard (activated during incidents)

- Container logs (CloudWatch Insights queries)
- Distributed trace waterfall (X-Ray)
- Network flow analysis (VPC Flow Logs → Athena)

15. Scenario 11: GenAI / Modern Data Infrastructure on AWS {#15-scenario-11-genai-data-infrastructure}

NEW SCENARIO — Caylent is increasingly focused on AI/ML. If the session tilts toward a "modern data/GenAI cloud infrastructure build," this is your playbook.

Prompt: "Design a GenAI-powered platform on AWS — the client wants to build internal AI applications using foundation models, with a focus on RAG (Retrieval-Augmented Generation) and custom model fine-tuning."

Discovery Questions

"GenAI infrastructure is evolving rapidly. Let me understand the scope:

Use cases: Customer support chatbot? Internal knowledge search?

Code assistant? Content generation? Each has different requirements.

Data: What data will feed the RAG pipeline? PDFs, databases, wikis?

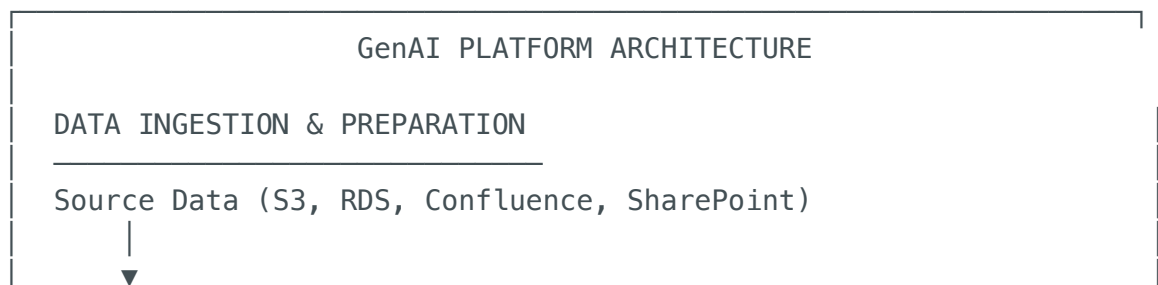
Privacy: Can data leave the AWS boundary? Or must everything stay in-region (rules out some SaaS options)?

Scale: How many concurrent users? Latency requirement for inference?

Team: Do they have ML engineers, or are they looking for no-code/low-code?

Budget: GPU inference is expensive — what's the monthly target?"

Architecture: GenAI Platform on AWS



Step Functions Orchestration:

1. Extract: Lambda → pull documents from sources
2. Transform: Lambda → chunk text, clean, normalize
3. Embed: SageMaker Endpoint (embedding model: Titan/Cohere)
4. Store: Amazon OpenSearch Serverless (vector store)
OR Amazon Aurora with pgvector extension

RAG INFERENCE PIPELINE

User Query



API Gateway (WebSocket for streaming responses)



Lambda / ECS (RAG Orchestrator)

- 1. Embed query (SageMaker or Bedrock)
- 2. Semantic search (OpenSearch / pgvector)
- 3. Build prompt (query + retrieved context + guardrails)
- 4. Call LLM (Amazon Bedrock – Claude, Llama, etc.)



Response → Guardrails for AI (content filtering, PII masking)



User (streaming response via WebSocket)

MODEL FINE-TUNING (if needed)

Training data (S3, curated datasets)



SageMaker Training Jobs (GPU: ml.p4d.24xlarge or ml.g5)



SageMaker Model Registry (versioning, approval workflow)



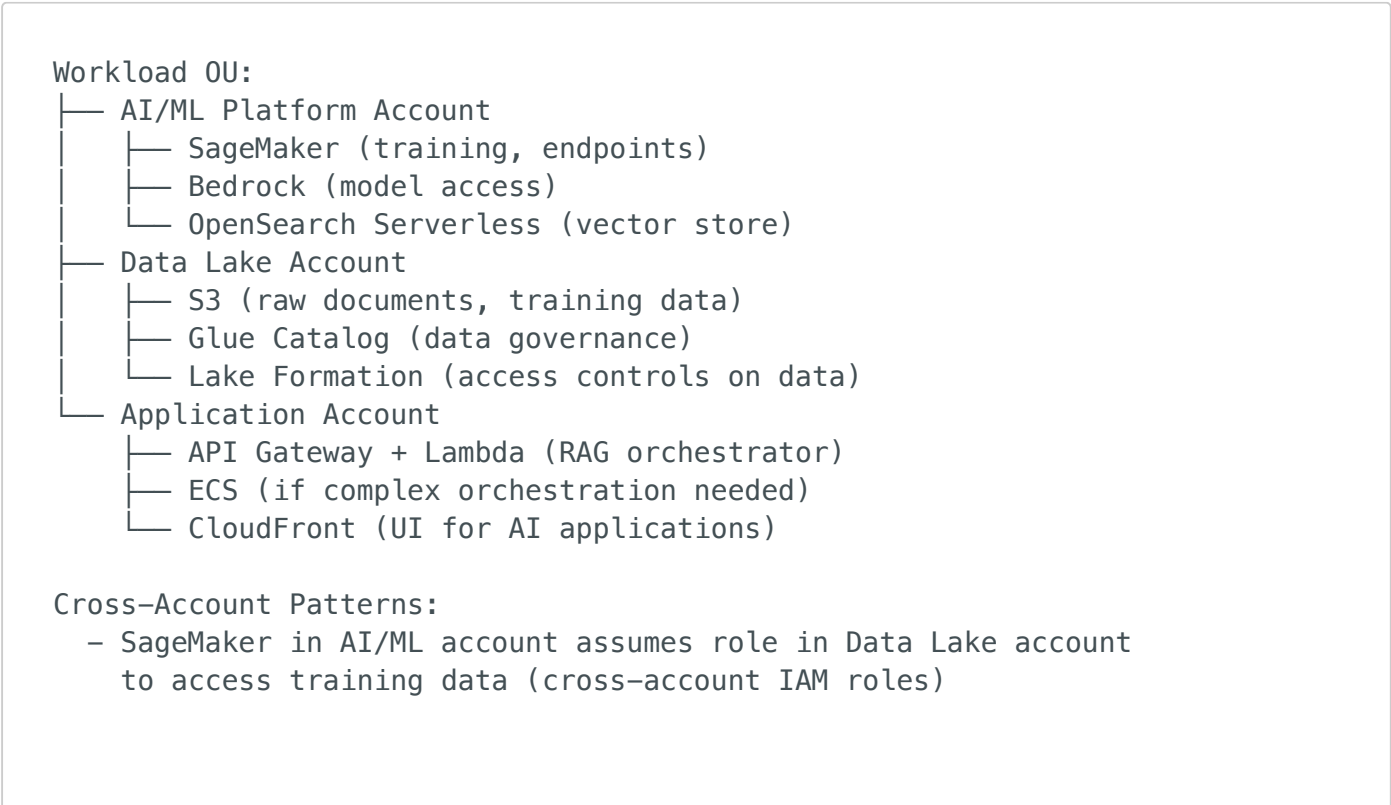
SageMaker Endpoint (real-time) or Batch Transform (batch)

Key Design Decisions

Decision	Choice	Trade-Off Rationale
Foundation Model	Amazon Bedrock (Claude 3.5 / Llama 3)	"Bedrock gives us managed access to multiple foundation models without managing GPU infrastructure. The trade-off vs SageMaker endpoints: less customization but 90% less operational overhead. I'd start with Bedrock and

Decision	Choice	Trade-Off Rationale
		move to self-hosted only if latency or cost requires it at scale."
Vector Store	OpenSearch Serverless	"OpenSearch Serverless handles the vector similarity search with zero infrastructure management. The trade-off vs pgvector: OpenSearch scales better for large vector collections (>10M documents), but pgvector is simpler if they already have Aurora PostgreSQL. I'd choose based on data volume."
Orchestration	Step Functions + Lambda	"For the RAG pipeline, Lambda handles the orchestration. If the workflow becomes complex (multi-turn conversations, tool use), I'd consider LangChain on ECS. But starting simple is the right call."
Guardrails	Amazon Bedrock Guardrails	"Content filtering, PII detection, and topic restrictions — managed by AWS. This is critical for enterprise adoption: the CISO needs to know AI responses won't leak sensitive data."

Account Structure for GenAI



- Application account calls Bedrock via VPC endpoint (traffic stays on AWS backbone, never internet)

Day 2: AI Ops

Model Monitoring:

- SageMaker Model Monitor: detect data drift, bias drift
- Custom CloudWatch metrics: response latency, token usage, cost per query
- EventBridge → Lambda: alert if model accuracy drops below threshold

Cost Management (critical for GenAI):

- Bedrock: track token usage per API key / per team
- SageMaker: auto-scaling inference endpoints (scale to zero when idle)
- Budget alerts: GenAI spend is unpredictable, set aggressive alerts

Auto-Remediation:

- Token usage spike → EventBridge → Lambda → throttle API key
- Model endpoint unhealthy → Lambda → rollback to previous model version
- Vector store index health → Lambda → trigger re-indexing pipeline

16. Caylent-Specific & Consulting Questions {#16-caylent-specific-questions}

Q1: Why Caylent? What attracts you to a consulting role at an AWS Premier Partner?

Answer:

"Three things attract me to Caylent specifically:

1. **AWS Premier Partner status** means you're working on the most complex, enterprise-scale AWS engagements. After 22 years of building systems, I want to apply my experience across multiple industries and challenging problems — consulting gives me that variety.

- 2. **Cloud-native focus with AI-driven operations.** Caylent isn't doing lift-and-shift factories — you're building modern architectures with automated Day 2 operations. That aligns with where I see the industry going: infrastructure that self-heals, auto-remediates, and reduces operational toil through intelligent automation.
 - 3. **Impact multiplier.** As a Principal Architect, I can shape how multiple client organizations adopt AWS — building reusable Terraform modules, reference architectures, and automated remediation patterns that scale across engagements. In a product company, I'd optimize one system. At Caylent, I can influence dozens."
-

Q2: Describe a time you led a discovery session with a client who had conflicting requirements

Answer:

"On a recent engagement, the client's CTO wanted multi-cloud for vendor independence, while the VP of Engineering wanted AWS-only for faster delivery. The security team needed SOC2 compliance regardless.

My approach (Cornerstone 1 — Own the Ambiguity):

- 1. **Facilitated a structured workshop** — I drew a decision matrix on the whiteboard:

Factor	Multi-Cloud	AWS All-In
Time to market	6-9 months	3-4 months
Operational complexity	High (2 clouds)	Low (1 cloud)
Vendor lock-in risk	Low	Medium
Team skills gap	Large (upskill needed)	Small (existing expertise)
SOC2 timeline	6+ months	3-4 months

- 2. **Proposed a middle ground:** AWS-only for the initial platform with **portability guardrails** — containerize on EKS (Kubernetes is portable), use Terraform (not CloudFormation), keep business logic cloud-agnostic.

3. **Outcome:** Both executives agreed. We shipped in 4 months on AWS. The portable architecture gave the CTO confidence that multi-cloud expansion was feasible without a rewrite.

Key lesson: The architect's job isn't to choose sides — it's to reframe the conversation around trade-offs and find the path that satisfies the most constraints."

Q3: How do you handle scope creep in a consulting engagement?

Answer:

"Scope creep is the #1 risk in consulting. I handle it through a structured change control process:

1. **SOW baseline:** Every engagement starts with clearly defined deliverables, timelines, assumptions, and exclusions.
 2. **Change request process:** When a client asks for something out of scope, I acknowledge it positively ('Great idea'), then assess impact:
 - Effort estimate (hours/days)
 - Impact on timeline
 - Impact on cost
 3. **Present trade-offs:** 'We can absolutely add this. It will take an additional 2 weeks and \$X. Alternatively, we can swap it for a lower-priority item already in scope.'
 4. **Document everything:** Change requests logged, approved by both parties, SOW amended.
 5. **Weekly status reports:** I track scope items as 'in-scope', 'change request', or 'backlog' so the client always has visibility."
-

Q4: How do you estimate effort for an architecture engagement?

Answer:

"I use a **T-shirt sizing** approach for initial estimates, then refine:

Phase	Activities	Typical Duration
Discovery	Stakeholder interviews, current state assessment	2-3 weeks
Design	Architecture design, PoC, HLD/LLD documents	3-4 weeks
Build	IaC implementation, CI/CD setup, security hardening	6-12 weeks
Migrate/Deploy	Data migration, application deployment, cutover	4-8 weeks
Optimize	Performance tuning, cost optimization, handoff	2-4 weeks

Estimation technique:

- 1. Break into work packages (not hours)
- 2. T-shirt sizes: S=1 week, M=2 weeks, L=4 weeks, XL=8 weeks
- 3. Add 20% buffer for unknowns
- 4. Identify risk items that could blow up estimates
- 5. Present as a range: 'We estimate 14-18 weeks, with the primary variable being Oracle migration complexity'"

Q5: Describe your experience with pre-sales support

Answer:

"As a Principal Architect, pre-sales is part of the role:

- 1. **Solution architecture for proposals:** Client RFP → I design the technical solution, create architecture diagrams, write the technical approach section, and estimate effort.
- 2. **Technical deep-dives with prospects:** Join sales calls to answer architecture questions, build trust with the client's technical team.

3. **Reference architectures:** I've built reusable reference architectures for common patterns (e-commerce on EKS, data lake, serverless API) that the sales team uses in proposals.
 4. **PoC execution:** When a prospect needs proof, I lead 2-week PoCs — 'show, don't tell.'
 5. **Win rate impact:** My involvement in pre-sales typically increases win rates because clients trust architects who can whiteboard their solution on the spot."
-

17. Stakeholder & Leadership Scenarios

{#17-stakeholder-leadership-scenarios}

Q6: A client's team is resistant to adopting IaC (Terraform). How do you drive adoption?

Answer:

"Resistance usually comes from fear, not dislike. My approach:

1. **Understand the resistance:** Job security fears? Skill gaps? Fear of breaking production?
2. **Start with a quick win:** Pick something low-risk:
 - 'Let's automate your dev environment provisioning. Currently 2 days. With Terraform, 15 minutes.'
 - This creates an internal champion.
3. **Pair programming:** I sit with their engineers, write Terraform together. Show, don't lecture.
4. **Prove with metrics:**
 - Before: 2 days to provision → After: 15 minutes
 - 'We eliminated 3 configuration drift incidents last month'

5. **Create guardrails, not gates:** Provide approved Terraform modules, CI/CD with **terraform plan** on PRs, Checkov scans.
 6. **Document and celebrate:** Write up the success story. Give credit to the team."
-

Q7: How do you mentor architects and engineers?

Answer:

"I follow a **tiered mentorship model**:

For Junior/Mid Engineers:

- Weekly 1:1s focused on growth
- Code review with educational comments ('here's why', not just 'fix this')
- Assign stretch projects with coaching guardrails

For Senior Engineers → Architect Track:

- Architecture decision ownership: 'You write the ADR, I'll review'
- Client-facing exposure: bring them to architecture workshops
- AWS certifications (SAP, DevOps Engineer)
- Cross-project rotation for breadth

For the Team:

- Architecture Guild: bi-weekly meetup — discuss patterns, review designs
- Blameless post-mortems: learning from failures
- Brown bag sessions: team members present learnings

Measuring success: 3 engineers earned AWS SAP certification within 6 months of my mentorship program."

Q8: Tell me about a time you made a wrong architectural decision

Answer:

"I initially chose **Apache Kafka (MSK)** for our event-driven architecture based on familiarity. After 3 weeks:

The problem:

- 12 event types, ~1000 events/second
- Team spent more time managing Kafka than building features
- MSK cost: \$2,400/month just for the cluster

My mistake: I chose based on past experience, not actual requirements.

How I handled it:

1. **Acknowledged openly:** 'I made a wrong call. Here's why and what I propose instead.'
2. **Proposed the fix:** EventBridge + SQS — managed, serverless, pay-per-event
3. **Showed the math:** EventBridge: ~100/month vs MSK :2,400/month
4. **Led the migration:** 1 week (event contracts unchanged, just transport)
5. **Wrote an ADR:** When Kafka IS right (>100K events/sec, replay needs, complex stream processing)

Lesson: Match the technology to the requirements, not the resume."

18. Technical Deep-Dive Rapid Fire {#18-technical-deep-dive-rapid-fire}

AWS Services

Q: Lambda cold start — how do you minimize it?

- Provisioned Concurrency (keep N instances warm)
- Keep deployment package small (<50MB, use layers for shared deps)
- Avoid VPC placement unless necessary (adds ENI attachment: ~6-8s)
- Use ARM/Graviton runtime (10% faster cold start, 20% cheaper)
- Initialize SDK clients outside handler (reused across invocations)

- SnapStart (Java — snapshot at init, restore in ~200ms)

Q: API Gateway — REST API vs HTTP API?

Feature	REST API	HTTP API
Cost	\$3.50/million	\$1.00/million
Latency	~29ms	~10ms
Features	Full (caching, WAF, usage plans, API keys)	Limited (JWT auth, CORS)
Use when	Need caching, WAF, or API keys	Simple proxy, cost-sensitive

Q: DynamoDB — when to use single-table design?

- Single-table: Relational-like access patterns needing sub-ms latency. Uses GSIs with overloaded keys.
- Multi-table: Independent entities, different teams, team lacks DynamoDB expertise.
- "Start multi-table. Move to single-table when access patterns are proven."

Q: ECS vs EKS?

Factor	ECS	EKS
Simplicity	Simpler, AWS-native	Complex, more features
Portability	AWS-only	Multi-cloud (K8s portable)
Ecosystem	Limited	Rich (Helm, Istio, ArgoCD, KEDA)
Team	AWS ops team	K8s expertise required
Cost	No cluster fee	\$0.10/hr/cluster
Choose	Small team, AWS-only, <20 services	Large team, K8s ecosystem needed

"I'd frame this as: ECS for simplicity, EKS for portability and ecosystem. At Caylent, I'd expect most enterprise clients lean toward EKS because they want the Kubernetes ecosystem and potential multi-cloud portability."

Q: Karpenter vs Cluster Autoscaler?

Feature	Cluster Autoscaler	Karpenter
Provisioning speed	Minutes (ASG-based)	Seconds (direct EC2 API)
Right-sizing	Pre-defined node groups	Any instance type, auto-selected
Consolidation	No	Yes (bin-packs, removes underutilized nodes)
Spot handling	Basic	Native (diversified, fallback to on-demand)
Choose	Legacy clusters, simple needs	New clusters, cost optimization critical

"Always recommend Karpenter for new EKS clusters. It's the future of EKS autoscaling."

Q: S3 storage classes?

Standard	→ Frequently accessed (default)
Standard-IA	→ Access < 1x/month, 128KB min, 30-day min
One Zone-IA	→ Same but single AZ (non-critical, reproducible data)
Glacier Instant	→ Archive with millisecond access
Glacier Flexible	→ 1-5 min retrieval, or bulk (5-12 hours)
Glacier Deep	→ Cheapest, 12-48 hour retrieval
Intelligent-Tier	→ Auto-moves between tiers (best for unknown patterns)

IaC & DevOps

Q: Terraform state locking — how does it work?

- State in S3 bucket (versioning enabled for recovery)
- DynamoDB table for locking (partition key: LockID)
- `terraform plan/apply` writes lock → prevents concurrent modifications
- Released on completion or timeout
- `terraform force-unlock <ID>` for stuck locks (use carefully)

Q: Corrupted Terraform state — how do you recover?

1. Don't panic — S3 versioning should be enabled
2. `aws s3api list-object-versions --bucket my-state-bucket`
3. Restore previous version

4. `terraform state push terraform.tfstate`
5. `terraform plan` — should show no changes
6. Root cause: investigate (concurrent access? manual edit?)

Q: GitOps — pull vs push model?

Push (traditional): CI Pipeline → `kubectl apply` → Cluster

Problem: pipeline needs cluster credentials, no drift detection

Pull (GitOps — ArgoCD): Git commit → ArgoCD polls → syncs to cluster

ArgoCD runs **INSIDE** the cluster

Benefits: no external access, Git = source of truth, auto-heal drift

Observability

Q: Memory leak in containerized service?

1. Grafana: container RSS memory trending up (sawtooth = GC normal; climb = leak)
2. Alert: memory > 80% of limit for > 30 minutes
3. Investigate: `kubectl top pods`, exec for profiling
4. Root cause: heap dump (Java: `jmap`; Node: `--inspect`; Go: `pprof`)
5. Fix: code fix + resource limits as safety net

Q: CloudWatch Metrics vs Logs vs Traces?

- **Metrics:** Numeric time-series (CPU%, request count). Dashboards and alerting.
- **Logs:** Text/structured events. Debugging, audit. Higher volume/cost.
- **Traces:** End-to-end request path. Latency analysis, dependency mapping.
- **Relationship:** Metric alerts you → logs tell you what → traces show you where.

19. Questions to Ask the Interviewer

{#19-questions-to-ask}

About the Role

1. "What does a typical Caylent engagement look like for a Principal Architect? How long are engagements?"
2. "How is the team structured? Do Principals lead consultant teams, or work independently across clients?"
3. "What percentage of time is client-facing vs internal (pre-sales, mentoring, building frameworks)?"
4. "What's the most challenging engagement Caylent has delivered in the last year?"

About Caylent's AI/Ops Focus

5. "I've noticed Caylent is emphasizing AI-driven cloud operations. What does that look like in practice for client engagements?"
6. "Are clients asking for GenAI infrastructure more frequently? What patterns are you seeing?"
7. "How does Caylent engage with AWS on the AI/ML partnership side — any co-development or joint solutions?"

About Growth & Culture

8. "How does Caylent support continuous learning? Certification budget? Conference attendance?"
9. "What does the career path beyond Principal Architect look like?"
10. "How does Caylent engage with AWS — co-sell, joint architecture reviews, re:Invent participation?"

About Delivery

11. "What's the standard Caylent reference architecture or tech stack that teams start from?"
 12. "How do you handle knowledge transfer at the end of an engagement — making clients self-sufficient?"
 13. "What tools does Caylent use internally for project management and documentation?"
-

Quick Reference: What to Say at Key Moments

Opening (when they give you the prompt)

"Great scenario. Before I start designing, let me ask a few discovery questions — this is how I'd approach it with a real client. I want to make sure I'm solving the right problem."

Before Drawing

"Based on what you've told me, the key constraints are [X, Y, Z]. Let me set up the board — I'll start with the account structure and then work through the architecture left-to-right."

Making a Technology Choice

☑ over [Y] here because [specific constraint]. If the requirement were different — for example, if [alternative scenario] — I'd reconsider and go with [Y]. This is a trade-off between [dimension A] and [dimension B]."

Addressing Day 2

"Now let me show how this lives and breathes after we deliver it. The IaC is modularized in Terraform, observability is baked in from day one, and we have automated remediation patterns — like EventBridge triggering Lambda to auto-heal or auto-scale — so the client's ops team isn't firefighting."

Closing

"To summarize: this architecture addresses the client's [primary requirement] while balancing [constraint 1] and [constraint 2]. The multi-account structure gives us security and blast radius isolation. The IaC and automation ensure it's operationally

sustainable after the engagement ends. And the modernization roadmap provides a path to continued value."

Prepared by Antigravity AI | June 2026

Tailored for Caylent Principal Cloud Architect — Whiteboarding Round

Enhanced with 4 Cornerstones Framework, Trade-Off Speaking Patterns, and AI-Driven Operations